



School of Future Tech

Case Study Report

on

**Backend Development - Fitness Tracker &
Workout Planner**

by

Ishita Nakhawa

150096725213

INDEX

Sr. No.	Title
1.	Introduction
2.	Problem Statement Background
3.	Problem Statement Design
4.	Methods & Algorithm/Technology Applied
5.	Implementation Details
6.	Snapshots
7.	Conclusion
8.	Reference
9.	GitHub Link

1. Introduction

FitTrack is a full-stack fitness tracking web application built as a backend-focused academic project. It gives users a single platform to:

- Log and review workout sessions
- Track body measurements (weight, height, BMI) over time with progress photos
- Create and follow structured workout plans
- View weekly and monthly analytics summaries with interactive charts
- Receive push-based workout reminders via Firebase Cloud Messaging

The system is split into two parts:

- A Node.js + Express.js REST API backend using MVC architecture with MongoDB + Mongoose
- A React.js SPA frontend that consumes the backend over HTTP using the native fetch API

2. Problem Statement Background

Domain: Health & Fitness Technology

Problem: Fitness enthusiasts lack a single, structured platform to log workouts, track body metrics, follow plans, and measure progress over time. They need a reliable backend system that:

- Persists data reliably and securely across sessions
- Keeps all data private and strictly user-specific
- Computes derivative values like BMI automatically on the server
- Sends timely reminders to keep users consistent
- Provides aggregated weekly and monthly progress summaries without manual calculation

3. Problem Statement Design

3.1 System Architecture

The system follows a three-tier architecture:

Tier 1 — Presentation (React SPA)

A browser-based single-page application handles all user interaction. It communicates exclusively with the backend via HTTP/JSON requests. The authentication state (token, user ID, and name) is stored in localStorage. All routing is handled client-side using React Router.

Tier 2 — Application (Express.js REST API)

The backend server runs on Node.js with Express.js. It applies global middleware (CORS, JSON parsing) and then route-level middleware (JWT auth, validation, file upload) before reaching the appropriate controller. The server exposes 18 REST endpoints grouped into 6 functional areas.

Tier 3 — Data & Services (MongoDB + Firebase)

MongoDB stores all persistent data via Mongoose ODM. Firebase Admin SDK handles two additional services—Auth (user sync, token verification, custom token generation) and Cloud Messaging (push notifications).

The overall request flow is:

Browser → HTTP/JSON (port 5001) → Express Middleware Chain → Controller → MongoDB / Firebase Admin SDK → JSON Response → Browser

3.2 Database Schema Design

Users Collection

Field	Type	Constraint	Notes
name	String	required	Display name
email	String	required, unique	Login identifier
password	String	required	bcrypt hash, never plain text
height	Number	optional	In cm, used for BMI
firebaseUid	String	optional	Synced from Firebase Auth
fcmToken	String	optional	FCM device token for push reminders

timestamps	auto	—	createdAt, updatedAt
------------	------	---	----------------------

Workouts Collection

Field	Type	Constraint	Notes
user	ObjectId → User	required	Owner reference
title	String	required	e.g. "Morning Run"
type	String	required	strength / cardio / yoga / hiit / etc.
duration	Number	required	In minutes
caloriesBurned	Number	optional	
exercises	Array	optional	Subdocuments: name, sets, reps, weight (kg)
personalRecord	Boolean	default: false	PR flag
date	Date	default: now	
timestamps	auto	—	createdAt, updatedAt

Plans Collection

Field	Type	Constraint	Notes
title	String	required	
description	String	optional	
category	String	required	strength / weight loss / cardio / etc.
durationWeeks	Number	optional	
createdBy	ObjectId → User	required	Creator reference
schedule	Array	optional	Subdocuments: day (string), exercises (string array)
followers	Array of ObjectId	optional	Users following this plan
timestamps	auto	—	createdAt, updatedAt

Metrics Collection

Field	Type	Constraint	Notes
user	ObjectId → User	required	Owner reference
weight	Number	required	In kg
height	Number	required	In cm
BMI	Number	auto-set	Calculated server-side, never client-supplied
photoUrl	String	optional	Full URL to local progress photo
date	Date	default: now	
timestamps	auto	—	createdAt, updatedAt

3.3 Complete API Endpoint Map

Tag	Method	Endpoint	Auth	Description
Auth	POST	/api/auth/register	None	Register user, sync to Firebase, return JWT + Firebase token
Auth	POST	/api/auth/login	None	Login, sync Firebase if needed, return JWT + Firebase token
Workouts	POST	/api/workouts	JWT	Log a workout (validated)
Workouts	GET	/api/workouts	JWT	Get all workouts for logged-in user, sorted date desc
Workouts	GET	/api/workouts/:id	JWT	Get single workout (403 if not owner)
Workouts	PUT	/api/workouts/:id	JWT	Update workout (403 if not owner)
Workouts	DELETE	/api/workouts/:id	JWT	Delete workout (403 if not owner)
Plans	POST	/api/plans	JWT	Create a plan (validated).
Plans	GET	/api/plans	JWT	Get all plans from all users, sorted by createdAt desc.

Plans	GET	/api/plans/search?key word=	JWT	Case-insensitive search on title, description, category
Plans	GET	/api/plans/:id	JWT	Get a single plan with populated creator and followers.
Plans	POST	/api/plans/:id/follow	JWT	Add user to followers (400 if already following).
Metrics	POST	/api/metrics	JWT	Log weight and height, and BMI is auto-calculated server-side.
Metrics	GET	/api/metrics/user/:id	JWT	Get the user's metric history sorted by date ascending (403 if not owned)
Metrics	POST	/api/metrics/:id/photo	JWT or Firebase	Upload a progress photo via multipart/form-data.
Analytics	GET	/api/analytics/summary?period=	JWT	Weekly (7 days) or monthly (30 days) aggregated summary
Reminders	PUT	/api/users/fcm-token	JWT	Save the device FCM token to the user document.
Reminders	POST	/api/reminders/send	JWT	Send FCM push notification to the user's registered device

4. Methods & Algorithms / Technology Applied

4.1 Dual Authentication System

The app runs two parallel authentication mechanisms that work together seamlessly.

Mechanism 1 — Custom JWT (used on all protected routes)

On register or login, the system performs four steps in sequence. First, express-validator checks all input fields against defined rules before the controller runs. Second, bcryptjs hashes the password using a generated salt (10 rounds)—the hash is stored in MongoDB, and the plain-text password is never persisted. Third, jsonwebtoken signs a 7-day token using JWT_SECRET from the environment. Fourth, syncFirebaseUser() looks up the user in Firebase Auth by email—if found, saves the UID back to MongoDB; if not found, creates a new Firebase Auth user and saves the UID. A Firebase custom token is also generated and returned alongside the JWT.

The full response to the client includes: _id, name, email, token (JWT), firebaseUid, and firebaseToken.

Mechanism 2 — Firebase Token (photo upload endpoint only)

The `protectAny` middleware first attempts to verify the bearer token as a standard JWT. If JWT verification fails, it falls through to Firebase Admin's `verifyIdToken()`. This means the photo upload endpoint accepts both token types, giving flexibility for Firebase-native clients.

JWT Middleware (``protect``):

Reads the `Authorization: Bearer` header, verifies the JWT, finds the user in MongoDB by the decoded ID (password field excluded), attaches the user to `req.user`, and calls `next()`. Returns 401 on a missing, expired, or invalid token.

4.2 BMI Calculation

BMI is computed entirely server-side in the metrics controller — it is never accepted from the client:

- Formula: weight (kg) divided by height in square meters
- Stored as a number rounded to one decimal place
- The frontend color-codes the result for the user:

BMI Range	Category	UI Color
Below 18.5	Underweight	Blue
18.5 – 24.9	Normal	Green
25 – 29.9	Overweight	Orange
30 and above	Obese	Red

4.3 Analytics Aggregation

No external aggregation library is used. The analytics controller builds the summary in three steps using only Mongoose and in-process JavaScript:

Step 1: Calculate the start date—7 days back for weekly, 30 days back for monthly.

Step 2: Query all workouts for the logged-in user within that date window, then loop through them in a single pass—accumulating total minutes, total calories, a count of personal records, and a map of workout counts grouped by type (e.g., { Cardio: 2, Strength: 2 }).

Step 3: Query the most recent metric document for the user to get the latest weight and BMI values.

The response includes `period`, `from`, `to`, `totalWorkouts`, `totalMinutes`, `totalCalories`, `personalRecords`, `workoutsByType`, `latestWeight`, and `latestBmi`.

4.4 File Upload Pipeline

The photo upload flow works as follows:

The browser sends a FormData request without a manually set Content-Type header. The browser sets it automatically, including the multipart boundary that Multer needs to parse the request. Multer processes the file using memory storage—the file is held as a buffer in memory and never written to disk by Multer itself. Only image/* MIME types are accepted, and the file size limit is 5 MB.

The uploadPhoto utility then takes that buffer, generates a timestamped filename (with spaces replaced by underscores), writes it to the uploads/ directory using fs.writeFileSync, and returns a full HTTP URL. That URL is saved to the Metric document in MongoDB. Progress photos are served as static files by Express from the uploads/ directory.

The BASE_URL constant is exported as a named export from api.js and imported in the Metrics page to construct the img src attribute when displaying saved photos.

4.5 Push Notifications (Firebase Cloud Messaging)

Push notifications work as a two-step process:

Step 1—Register device: The client calls PUT /api/users/fcm-token with the device's FCM registration token. The server saves this token to the fcmToken field on the user document in MongoDB.

Step 2—Send reminder: The client calls POST /api/reminders/send with an optional custom message. The server looks up the user's saved FCM token, then calls messaging.send() from Firebase Admin SDK with a notification payload. The notification title is always "FitTrack Reminder," and the body defaults to "Time for your workout!" if no custom message is provided. If no FCM token has been saved, the endpoint returns a 400 error with instructions to register the token first.

Reminders are fully stateless—nothing is stored in the database. They are ephemeral push notifications only.

4.6 Input Validation Middleware

Express-validator rule sets are applied as route middleware before any controller runs. If any rule fails, the middleware returns 400 with a detailed errors array immediately—the controller function is never reached.

Rule Set	Applied To	Rules
registerRules	POST /api/auth/register	name: not empty · email: valid format · password: minimum 6 characters
loginRules	POST /api/auth/login	email: valid format · password: not empty

workoutRules	POST /api/workouts	title: not empty · type: not empty · duration: numeric
planRules	POST /api/plans	title: not empty · category: not empty
metricRules	POST /api/metrics	weight: numeric · height: numeric

4.7 Technology Stack

Layer	Technology	Version
Runtime	Node.js	LTS
Web Framework	Express.js	^5.2.1
Database ODM	MongoDB + Mongoose	^9.10.3
JWT Auth	jsonwebtoken	^9.0.3
Password Hashing	bcryptjs	^3.0.3
Firebase Integration	firebase-admin	^14.5.0
Input Validation	express-validator	^7.3.2
File Uploads	multer	^2.4.0
API Documentation	Postman	Browser
CORS	cors	^2.8.6
Environment Variables	dotenv	^18.0.5
Dev Auto-Reload	nodemon	^3.1.14
Frontend Framework	React	^19.3.0
Frontend Build	Vite	^8.3.2
Frontend Routing	react-router-dom	^7.18.4
Frontend Charts	Recharts	^3.10.1
HTTP Client	fetch API	native browser

5. Implementation Details

5.1 Backend Project Structure

```

fittrack-backend/
├── server.js           ← Entry point
├── .env               ← Environment variables
├── .env.example       ← PORT, MONGO_URI, JWT_SECRET
├── serviceAccountKey.json ← Firebase credentials
├── package.json
├──
├── config/
│   ├── db.js         ← MongoDB connection
│   └── firebase.js   ← Firebase Admin setup
├── middleware/
│   ├── authMiddleware.js ← JWT verification
│   ├── firebaseAuth.js  ← Firebase token verification
│   ├── uploadMiddleware.js ← File upload (multer)
│   └── validateMiddleware.js ← Input validation rules
├── models/
│   ├── User.js
│   ├── Workout.js
│   ├── Plan.js
│   └── Metric.js
├── controllers/
│   ├── authController.js
│   ├── workoutController.js
│   ├── planController.js
│   ├── metricController.js
│   ├── analyticsController.js
│   └── reminderController.js
├── routes/
│   ├── authRoutes.js
│   ├── workoutRoutes.js
│   ├── planRoutes.js
│   ├── metricRoutes.js
│   ├── analyticsRoutes.js
│   ├── userRoutes.js
│   └── reminderRoutes.js
├── utils/
│   └── uploadPhoto.js ← Saves photo, returns URL
├── uploads/           ← Stored progress photos
├── docs/
│   ├── Fittrack.postman_collection.json
│   └── FitTrack.postman_environment.json

```

File responsibilities:

- `server.js` — Entry point. Connects to MongoDB, initialises Firebase, applies global middleware (CORS, JSON), mounts all route modules, and starts the HTTP server.
- `config/db.js` — Calls `mongoose.connect()` with `MONGO_URI` from environment. Exits the process on failure.
- `config/firebase.js` — Initializes Firebase Admin SDK using `serviceAccountKey.json`. Exports `auth` and `messaging` instances.
- `middleware/authMiddleware.js` — `protect()` — JWT-only verification, attaches `req.user`.

- middleware/firebaseAuth.js—protectAny()—tries JWT first, falls through to Firebase token verification.
- middleware/uploadMiddleware.js—Multer configured with memory storage, image-only filter, and 5 MB limit.
- middleware/validateMiddleware.js — Five express-validator rule sets with a shared handleValidation handler.
- models/—Mongoose schemas for User, Workout, Plan, and Metric with timestamps: true on all.
- controllers—business logic. Each controller function is async, wraps logic in try/catch, and returns JSON responses.
- routes—Express routers. Wire HTTP methods and paths to middleware chains and controller functions.
- utils/uploadPhoto.js—Writes the multer memory buffer to the uploads/ directory and returns a full URL.
- uploads/—Static directory served by Express at /uploads. Stores progress photos.

5.2 Frontend Project Structure

```

fittrack-frontend/
├── package.json           ← Vite + React 19, Recharts, React Router v7
├── index.html             ← Vite entry HTML
├── src/
│   ├── main.jsx           ← App entry point (mounts React to DOM)
│   ├── App.jsx            ← Routes and Private wrapper
│   ├── App.css            ← Global styles
│   ├── index.css          ← Base CSS
│   ├── api.js             ← All API calls (fetch wrapper, BASE_URL)
│   ├── context/
│   │   └── ToastContext.jsx ← Global toast notifications
│   ├── components/
│   │   └── Navbar.jsx      ← Auth-aware navigation
│   └── pages/
│       ├── Login.jsx
│       ├── Register.jsx
│       ├── Dashboard.jsx   ← Stats cards, Recharts charts, recent workout
│       ├── Workouts.jsx    ← Log and delete workouts
│       ├── Plans.jsx       ← Create, search, follow plans
│       └── Metrics.jsx      ← Log metrics, BMI badge, photo upload

```

File responsibilities:

- `main.jsx` — The actual entry point of the Vite app. Mounts the React app to the DOM using `ReactDOM.createRoot()` and wraps it in `React.StrictMode`.
- `App.jsx` — Sets up `BrowserRouter`, wraps the app in `ToastProvider`, defines all routes with a `Private` wrapper component that redirects unauthenticated users to `/login`, and renders a `NotFound` component for unknown routes.
- `api.js` — Central HTTP helper. `BASE_URL` is hardcoded to `http://localhost:5001`. Provides a `myFetch` wrapper that injects the JWT from `localStorage` into every request header. Exports 13 API methods plus a special `uploadPhoto` method that uses raw `fetch` with `FormData`. Also exports `BASE_URL` as a named export for use in `Metrics.jsx`.
- `ToastContext.jsx` — Global notification system. Toasts auto-dismiss after 3500 ms with a 350 ms fade-out. Exposes `success()`, `error()`, and `info()` helpers via the `useToast()` hook.
- `Navbar.jsx`—Auth-aware navigation bar. Reads `localStorage` for token and name. Listens to `window.storage` and `window.Focus` events to stay in sync across tabs without a page reload.
- `Login.jsx`—Email and password form with client-side validation. Stores token, `_id`, and name to `localStorage` on success.
- `Register.jsx`—Four-field form (name, email, password, height). Validates all fields before submission—height must be 50–300 cm and password must be at least 6 characters.
- `Dashboard.jsx`—Fires three API calls in parallel on mount (weekly summary, all workouts, and all metrics). Displays 6 stat cards, a weight trend line chart, a workout duration bar chart (both via `Recharts`), and a recent workouts list.
- `Workouts.jsx`—Add and delete workout sessions. Offers 6 type options: Cardio, Strength, Flexibility, HIIT, Sports, and Other. Each workout is shown with a type initials badge and a delete button.
- `Plans.jsx`- Two-panel layout: create a plan on the left and search plans by keyword on the right. Displays all plans with a follow button and live follower count.
- `Metrics.jsx`—Log weight and height measurements. Each metric card shows the calculated BMI with a color-coded category badge, the entry date, and a hidden file input for uploading a progress photo.

5.3 Key Implementation Notes

Port configuration:

`api.js` hardcodes the base URL as `http://localhost:5001`. The backend defaults to port 5000 if `PORT` is not set. The `.env` file must contain `PORT=5001` for the frontend and backend to connect correctly.

Route ordering in plans:

The `/api/plans/search` route is registered before `/:id` in the router. This is intentional and critical—if reversed, Express would interpret the string "search" as a MongoDB ObjectId parameter and throw a cast error.

Parallel data loading on Dashboard:

The dashboard fires all three data requests simultaneously using `Promise.all`, reducing the total load time compared to sequential calls. A loading spinner is shown while pending, and an error banner is shown if any request fails.

Ownership enforcement:

Every endpoint that reads or modifies user-specific data checks that the resource belongs to the logged-in user before proceeding. A mismatch returns HTTP 403 Forbidden—not 404—so that the existence of the resource is not leaked to unauthorized callers. This applies to workout read, update, and delete, and to metric read and photo upload.

Auth state in localStorage:

Three values are stored after a successful login or registration—the JWT (sent as the `Authorization: Bearer` header on every API call), the MongoDB user `_id` (used to call the metrics endpoint), and the user's display name (shown in the Navbar and Dashboard greeting).

Cross-tab auth sync:

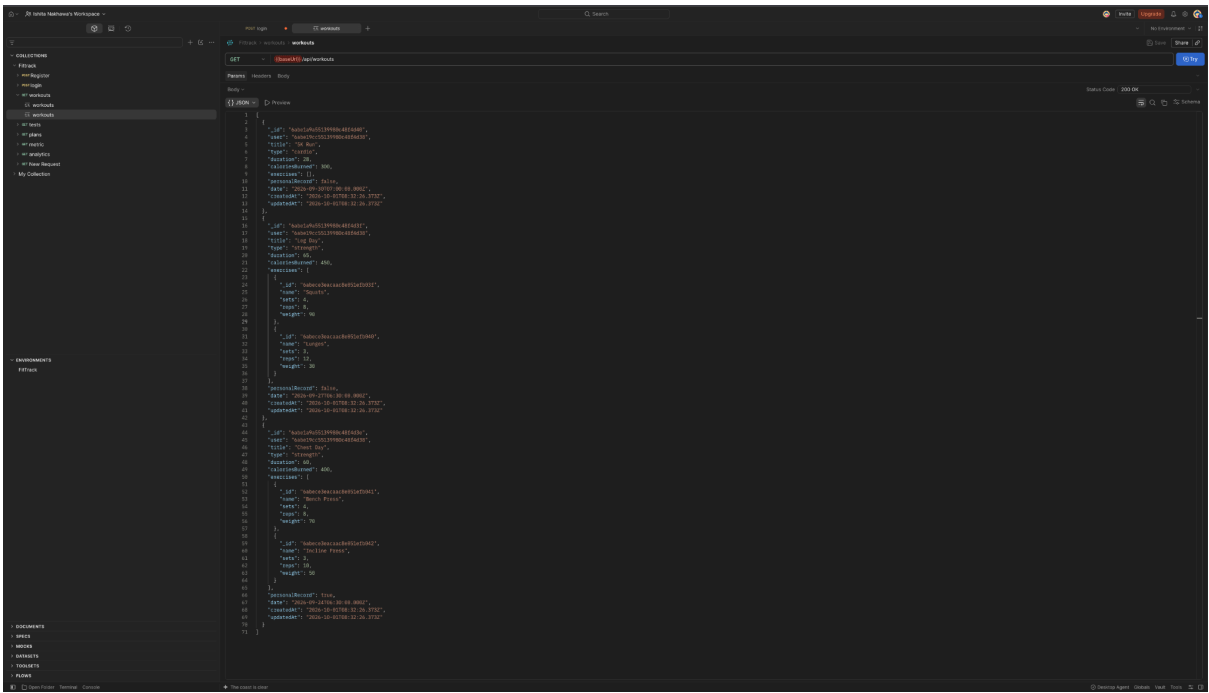
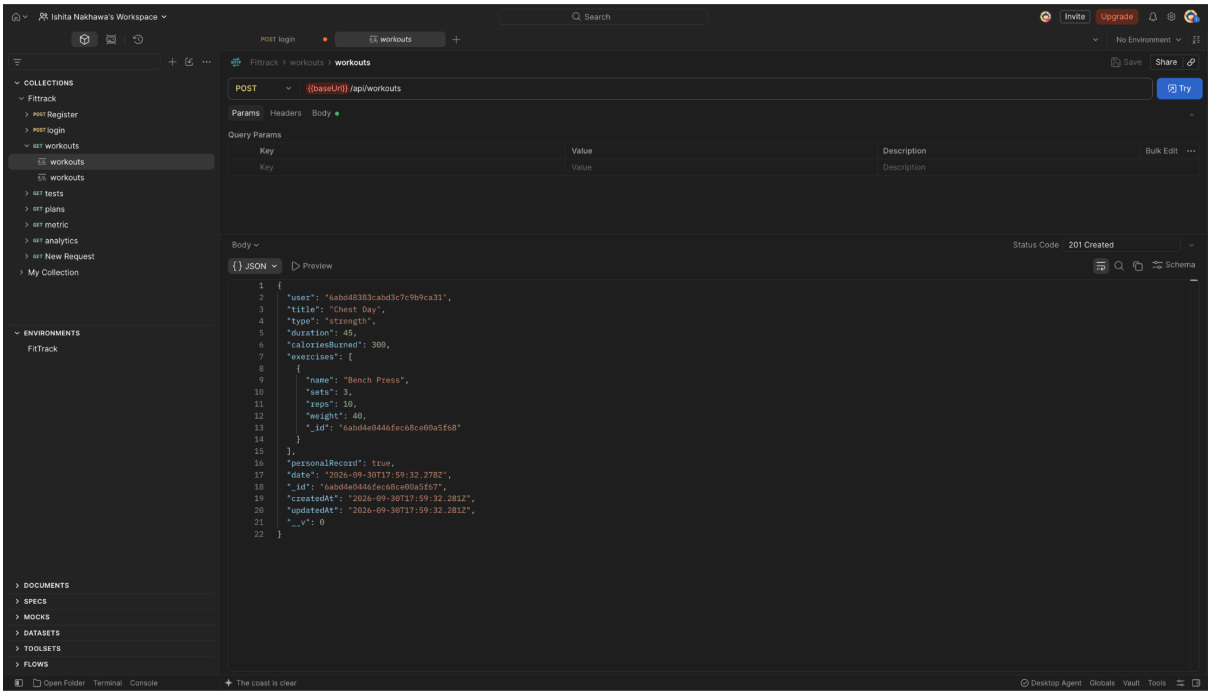
The Navbar listens to both the browser storage event (fires when another tab modifies localStorage) and the focus event (fires when the user returns to this tab) from another window). This keeps the navbar auth state accurate without requiring a page reload.

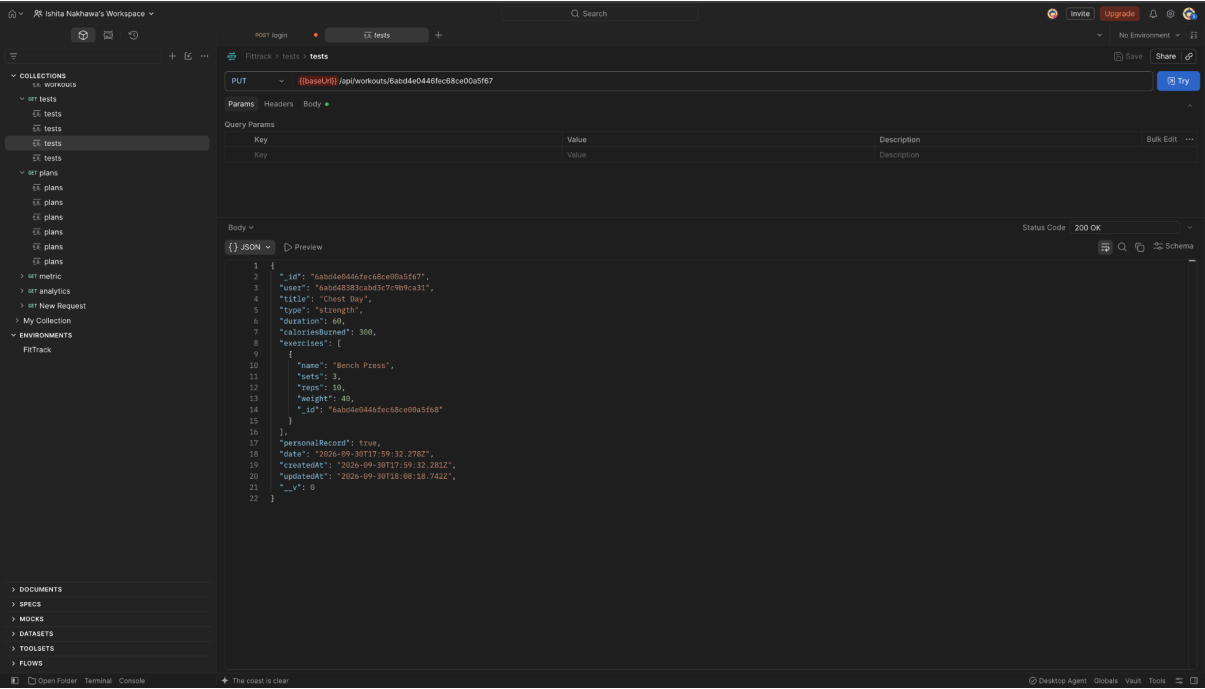
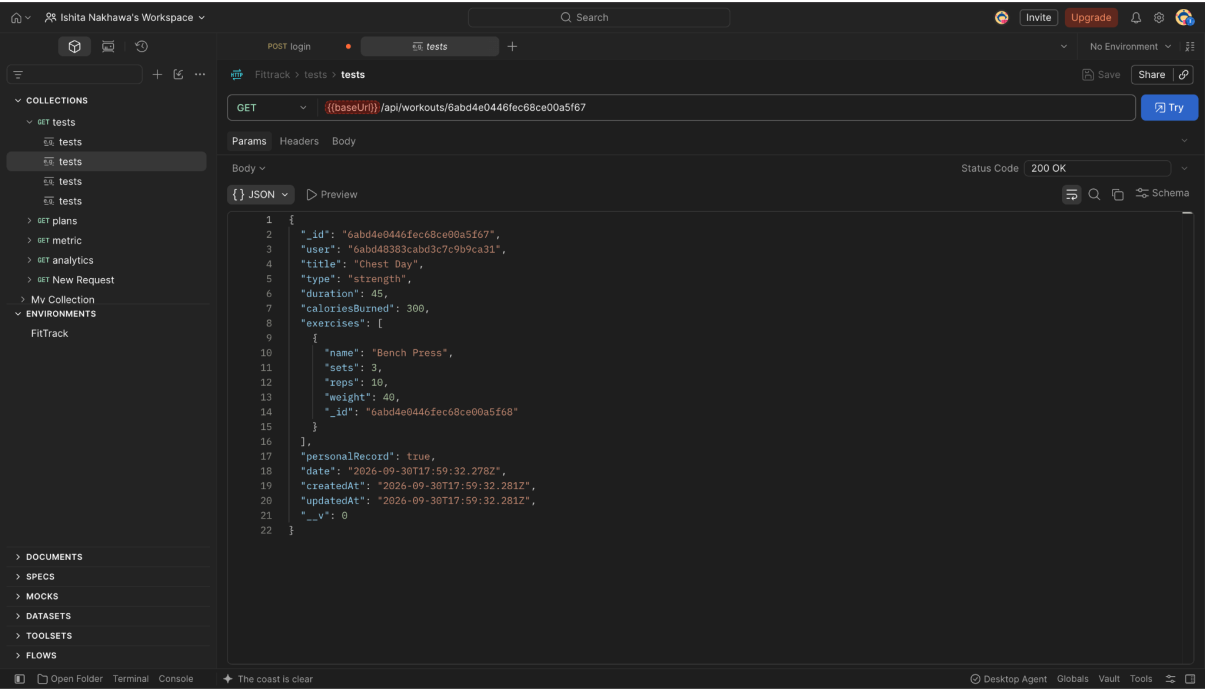
6. Snapshots

Postman Snapshots:

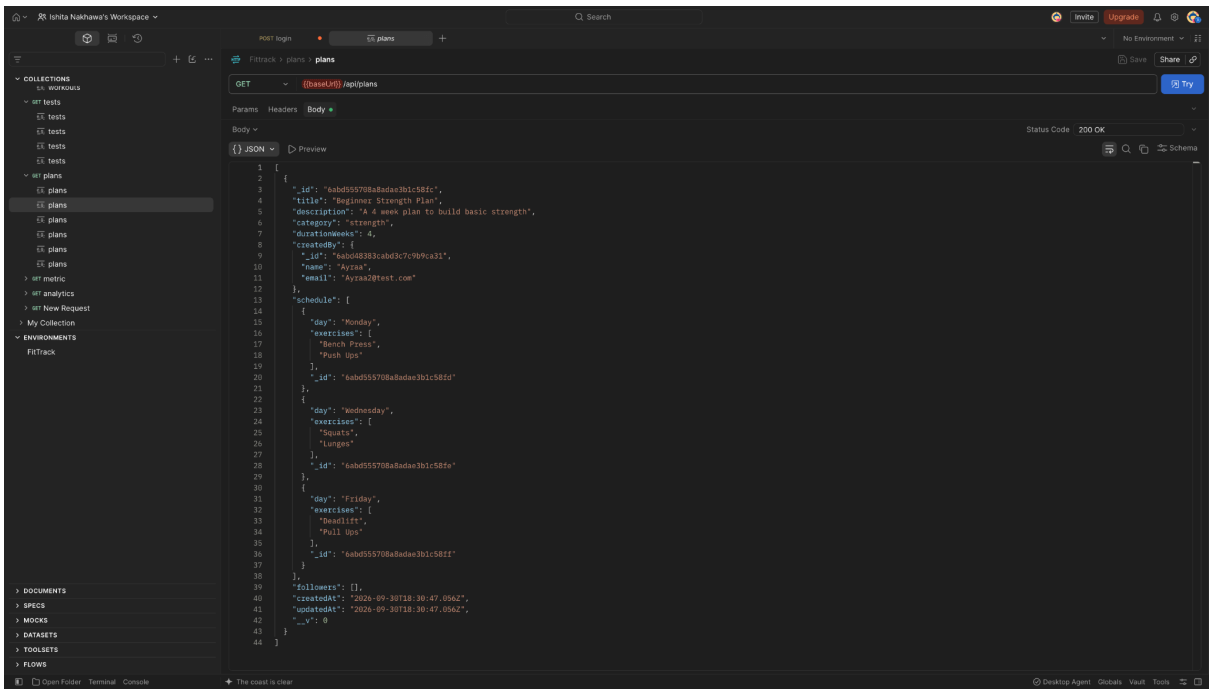
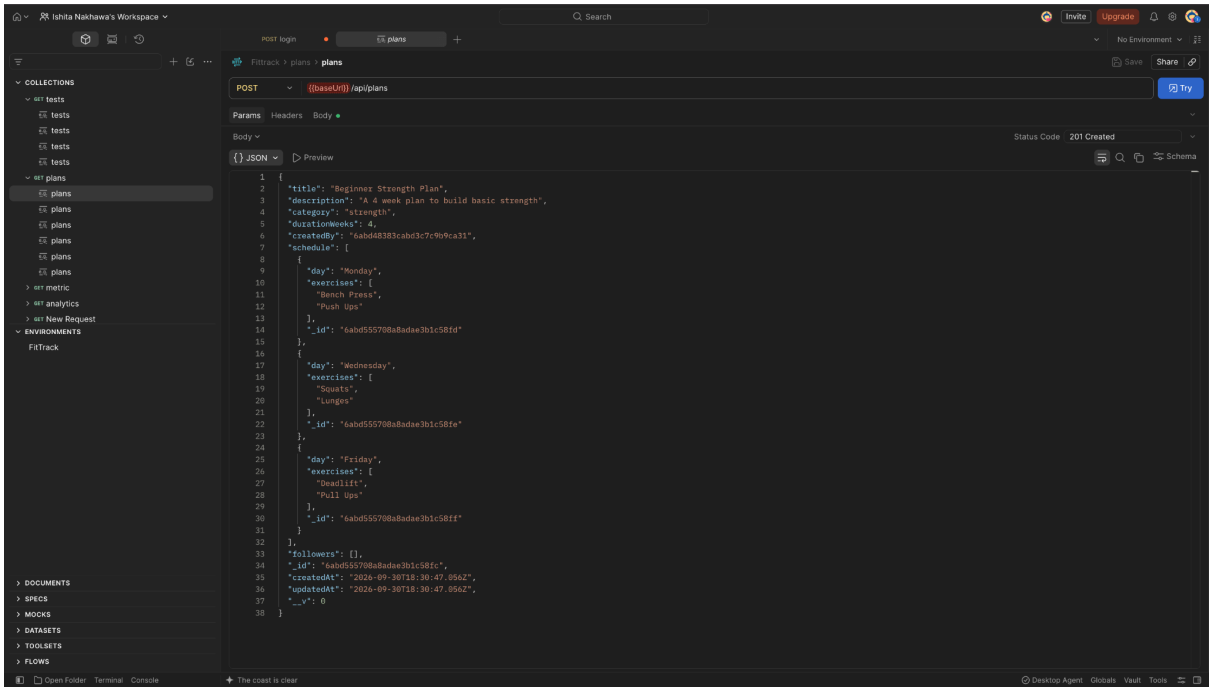
Register:

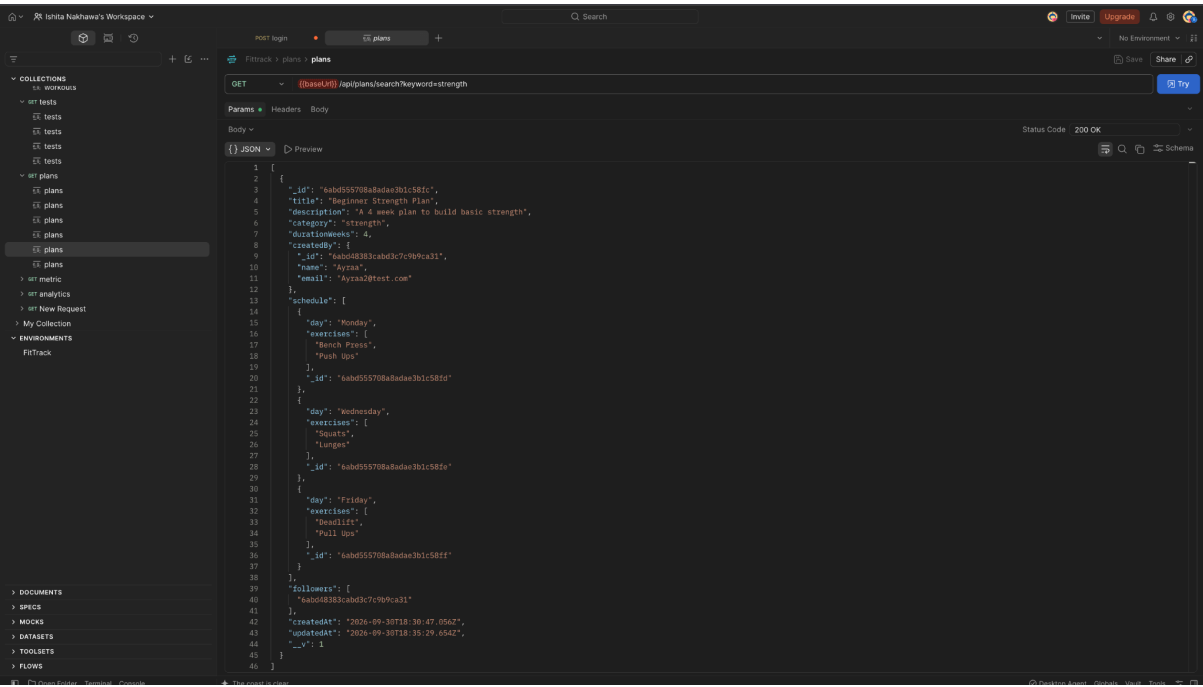
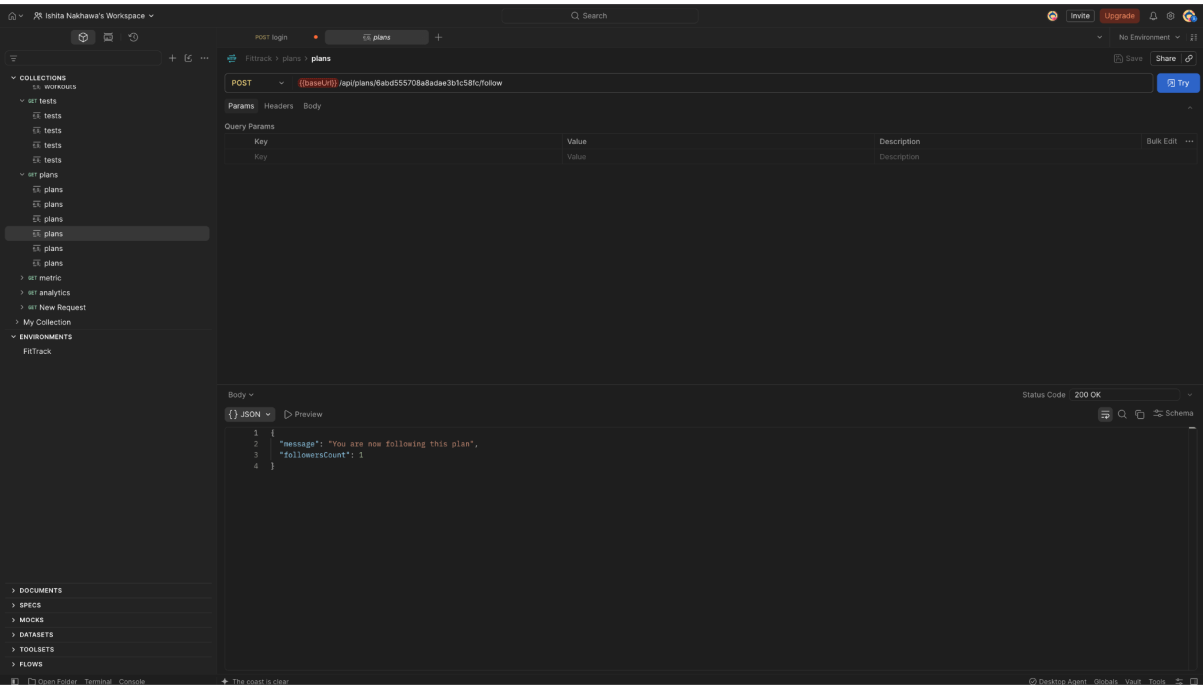




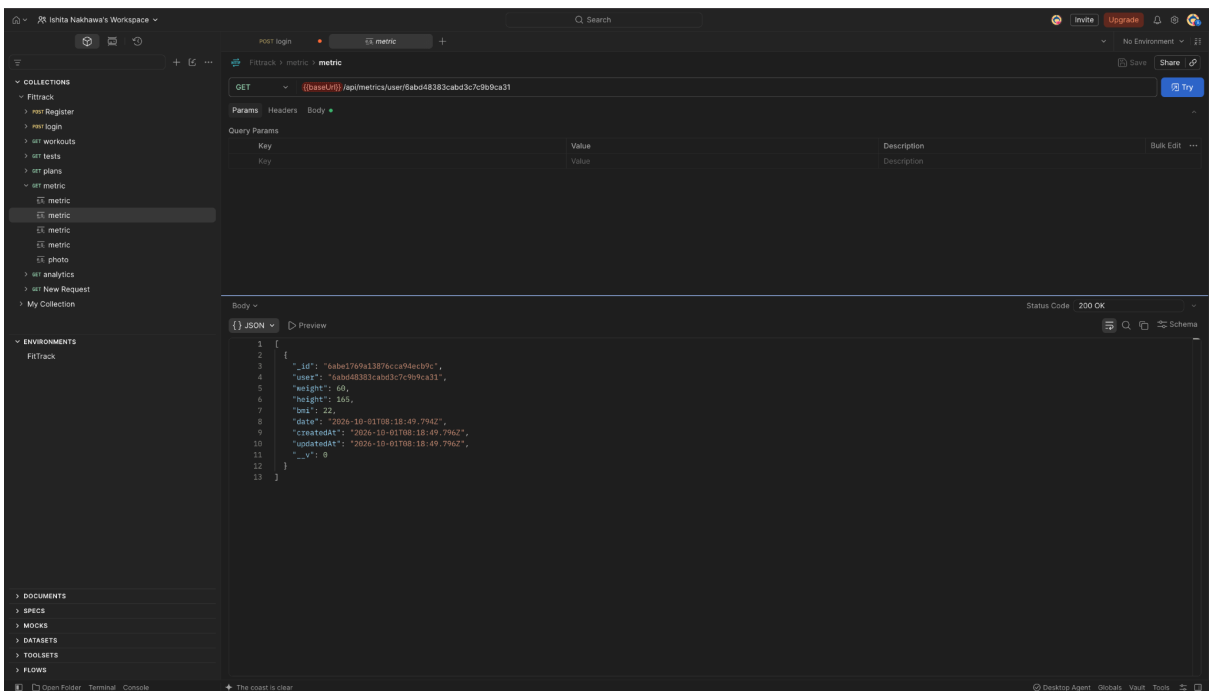
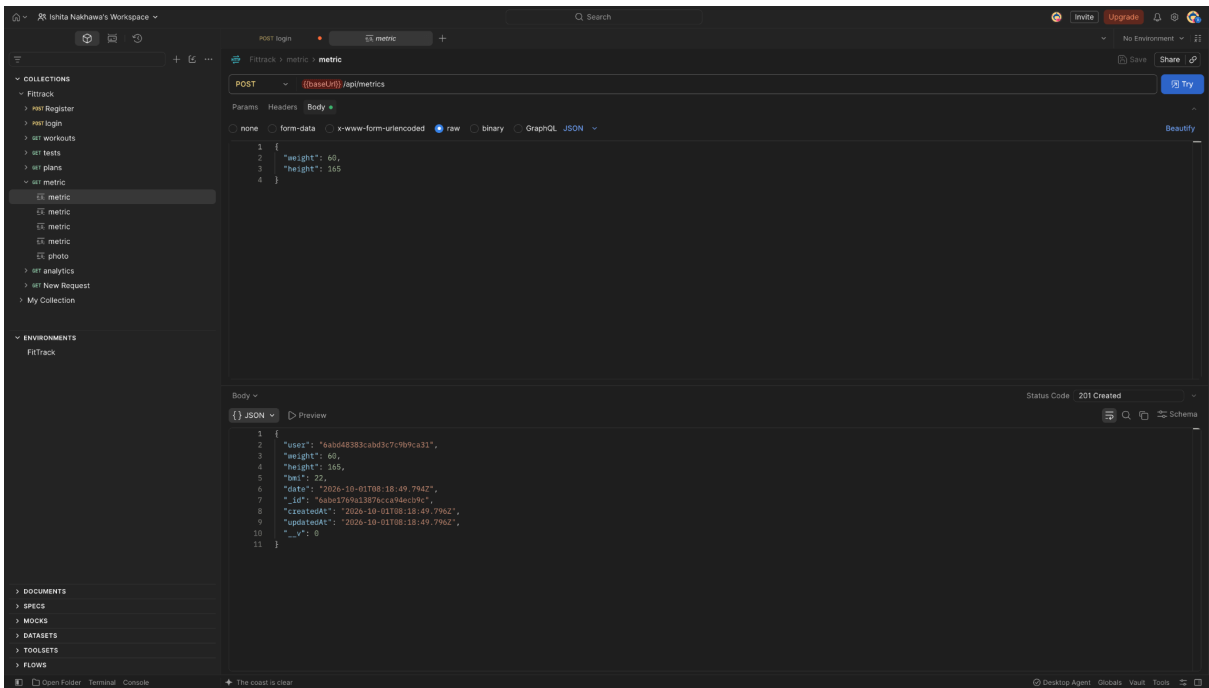


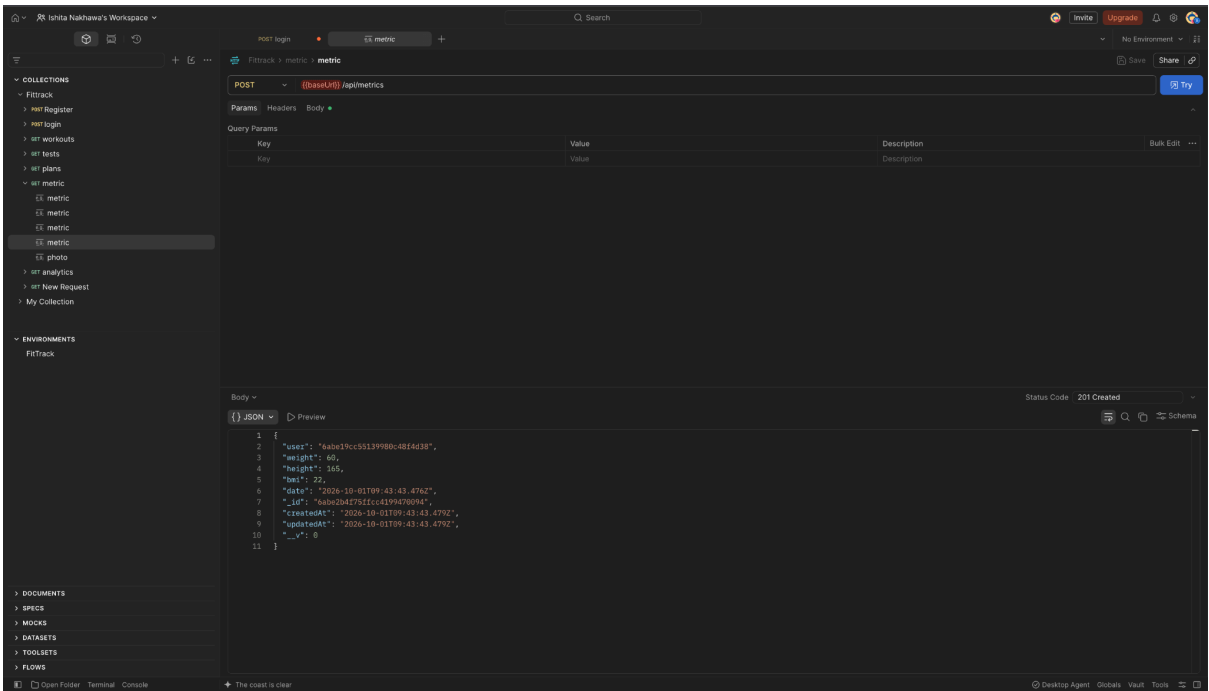
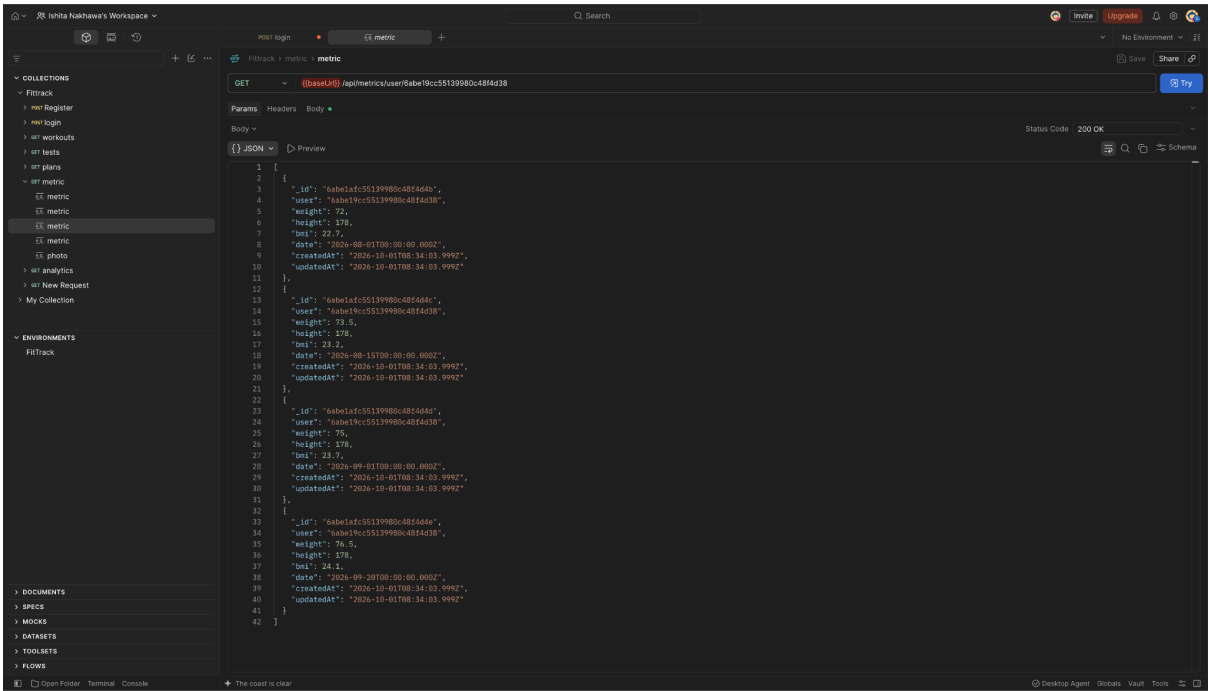
Plans:

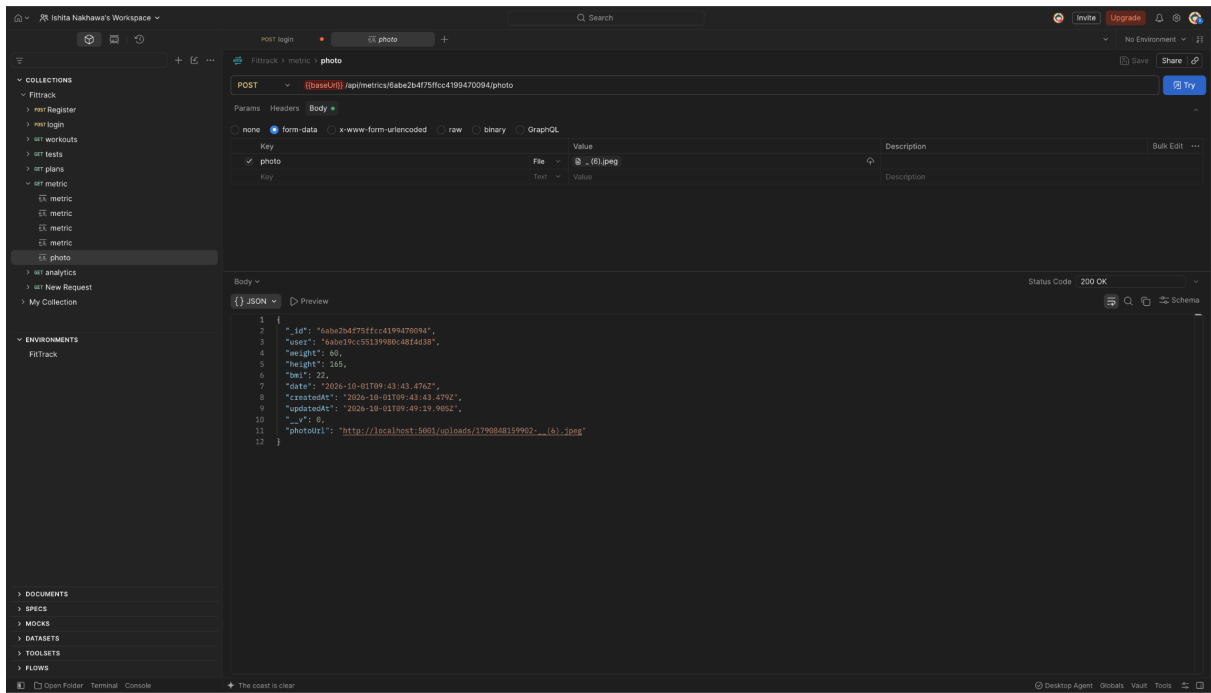




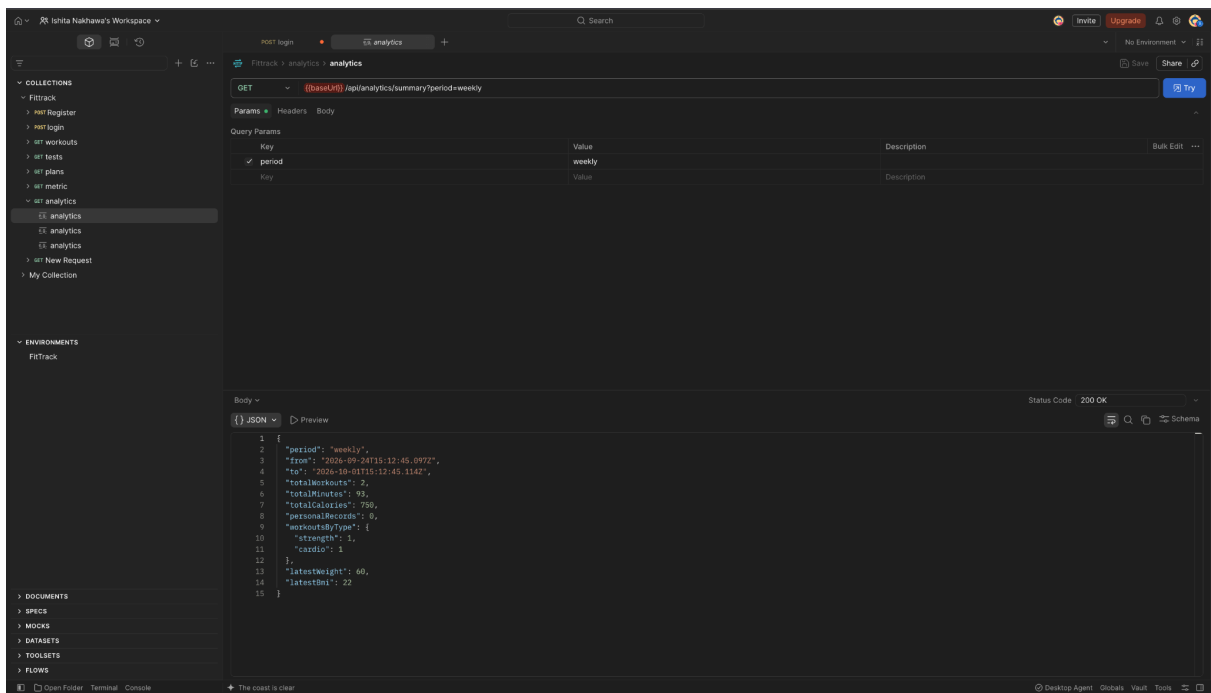
Metric:

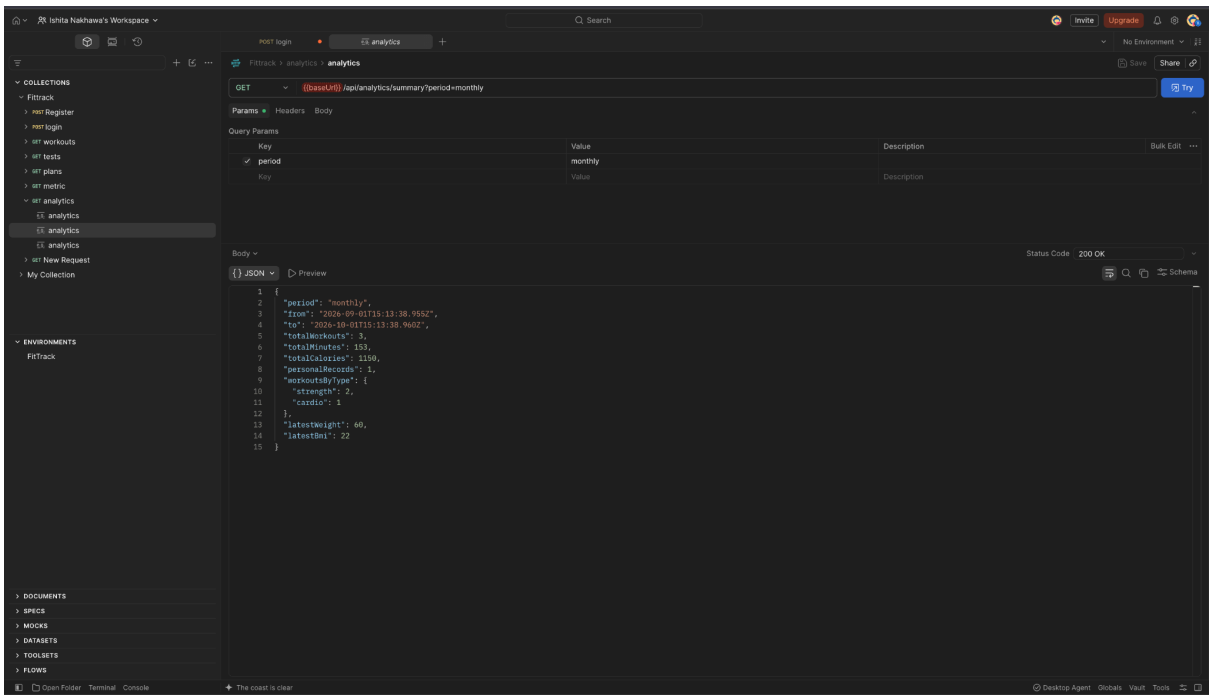




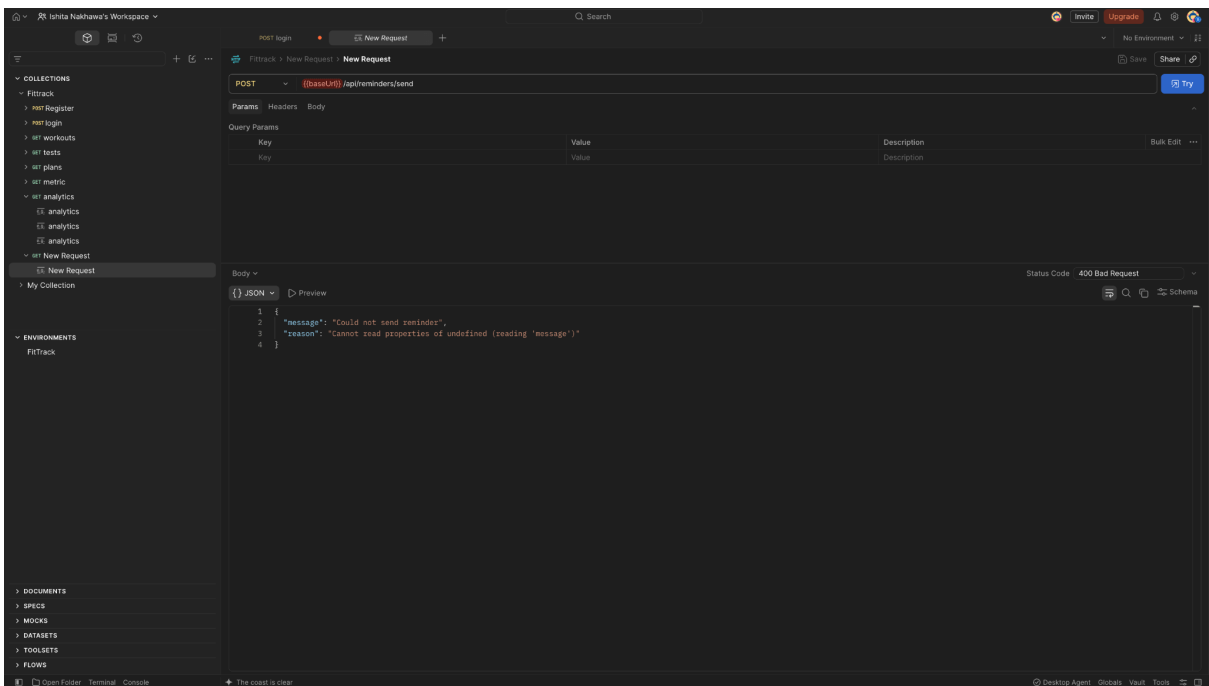


Analytics:

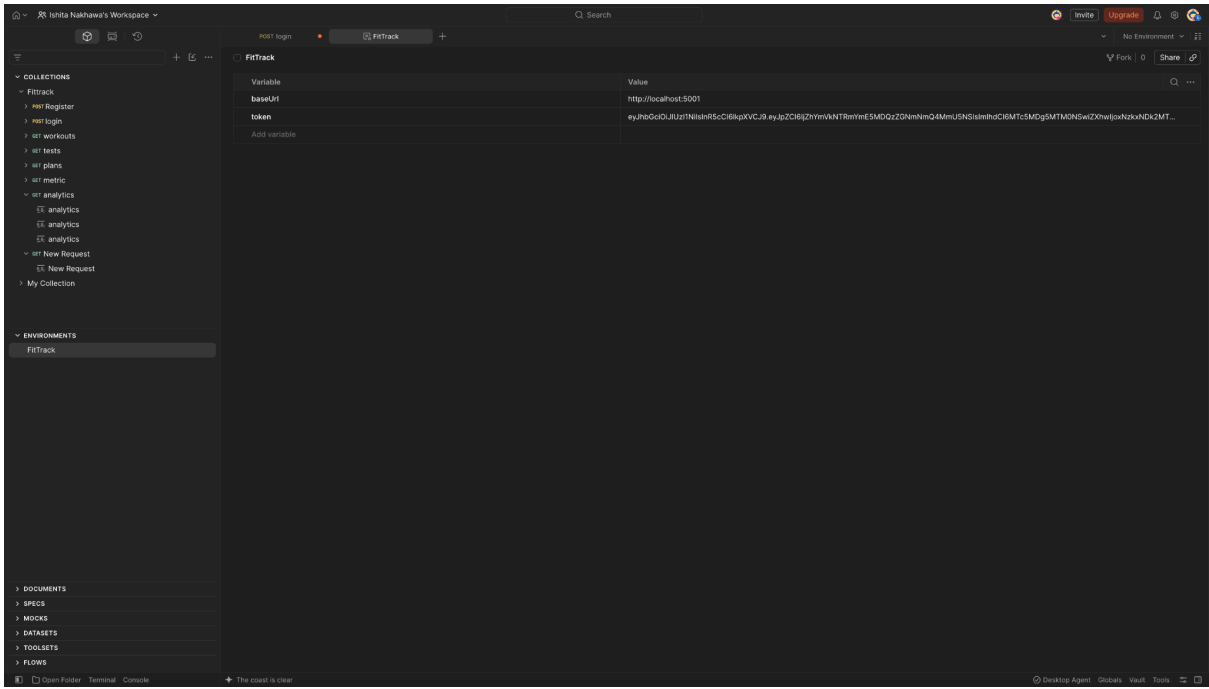




Reminder:

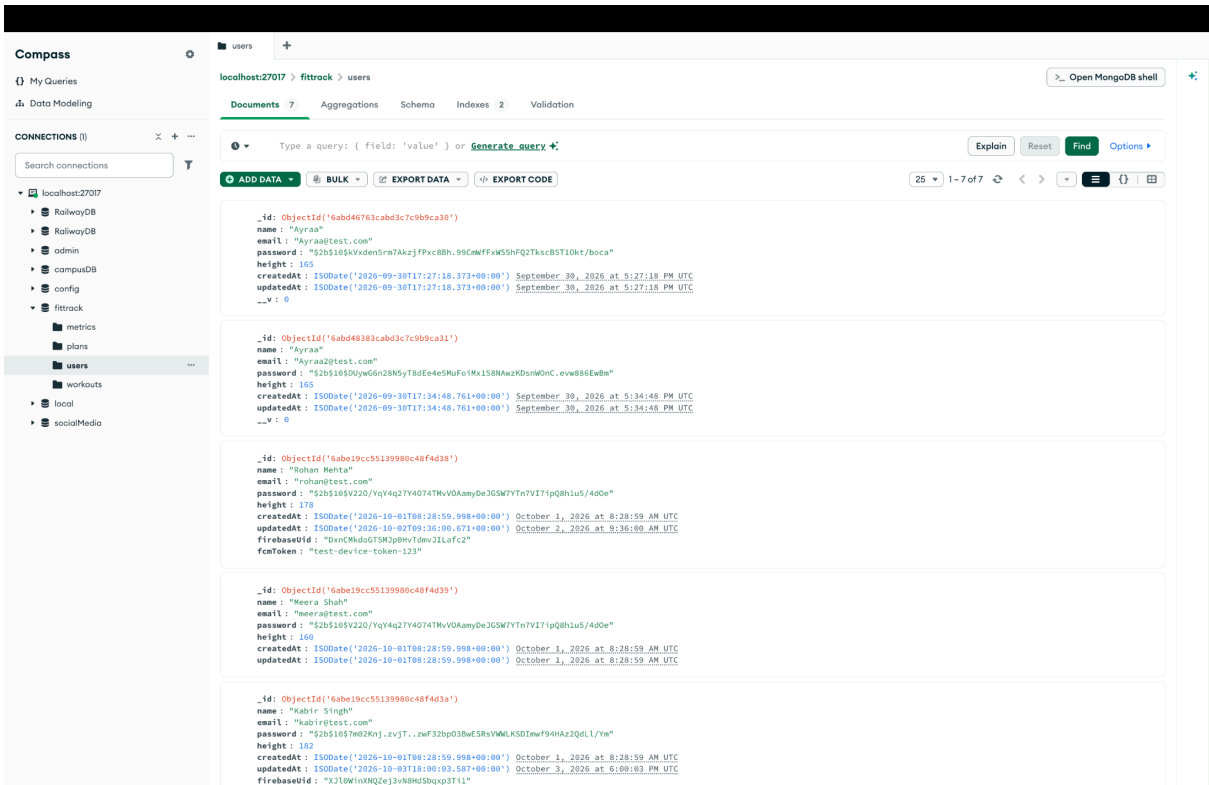


Environment:



MongoDB Snapshots

User:



Workout:

Compass

My Queries | Data Modeling

Connections (1) | localhost:27017

Documents | Aggregations | Schema | Indexes | Validation

Type a query: { field: 'value' } or [Generate Query](#)

ADD DATA | BULK | EXPORT DATA | EXPORT CODE

25 | 1 - 14 of 14

```
{
  "_id": ObjectId("6abe1a9a55139980c48f4d3b"),
  "user": ObjectID("6abd46763cabdc7c9b9ca3b"),
  "title": "Morning Run",
  "type": "cardio",
  "duration": 30,
  "caloriesBurned": 280,
  "exercises": Array (empty),
  "personalRecord": false,
  "date": ISODate("2026-09-25T07:00:00.000+00:00") September 25, 2026 at 7:00:00 AM UTC,
  "createdAt": ISODate("2026-10-01T08:12:26.373+00:00") October 1, 2026 at 8:12:26 AM UTC,
  "updatedAt": ISODate("2026-10-01T08:12:26.373+00:00") October 1, 2026 at 8:12:26 AM UTC
}
```

```
{
  "_id": ObjectId("6abe1a9a55139980c48f4d3b"),
  "user": ObjectID("6abd46763cabdc7c9b9ca3b"),
  "title": "Voga Flow",
  "type": "yoga",
  "duration": 40,
  "caloriesBurned": 150,
  "exercises": Array (empty),
  "personalRecord": false,
  "date": ISODate("2026-09-26T18:00:00.000+00:00") September 26, 2026 at 6:00:00 PM UTC,
  "createdAt": ISODate("2026-10-01T08:12:26.373+00:00") October 1, 2026 at 8:12:26 AM UTC,
  "updatedAt": ISODate("2026-10-01T08:12:26.373+00:00") October 1, 2026 at 8:12:26 AM UTC
}
```

```
{
  "_id": ObjectId("6abe1a9a55139980c48f4d3b"),
  "user": ObjectID("6abd46763cabdc7c9b9ca3b"),
  "title": "Back and Biceps",
  "type": "strength",
  "duration": 55,
  "caloriesBurned": 320,
  "exercises": Array (2),
  "personalRecord": true,
  "date": ISODate("2026-09-29T09:00:00.000+00:00") September 29, 2026 at 9:00:00 AM UTC,
  "createdAt": ISODate("2026-10-01T08:12:26.373+00:00") October 1, 2026 at 8:12:26 AM UTC,
  "updatedAt": ISODate("2026-10-01T08:12:26.373+00:00") October 1, 2026 at 8:12:26 AM UTC
}
```

```
{
  "_id": ObjectId("6abe1a9a55139980c48f4d3b"),
  "user": ObjectID("6abe19cc55139980c48f4d3b"),
  "title": "Chest Day",
  "type": "strength",
  "duration": 60,
  "caloriesBurned": 400,
  "exercises": Array (2),
  "personalRecord": true,
  "date": ISODate("2026-09-24T06:30:00.000+00:00") September 24, 2026 at 6:30:00 AM UTC
}
```

Plans:

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

localhost:27017

RailwayDB

RailwayDB

odmin

campusDB

config

fittrack

metrics

plans

users

workouts

local

socialMedia

users

workouts

plans

+

localhost:27017 > fittrack > plans

Open MongoDB shell

Documents 5

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#) *

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

25 1 - 5 of 5

```

{
  "_id": ObjectId("6abd555708a8adae3b1c58fc"),
  "title": "Beginner Strength Plan",
  "description": "A 4 week plan to build basic strength",
  "category": "strength",
  "durationWeeks": 4,
  "createdBy": ObjectId("6abd48383cabd3c7c9b9ca31"),
  "schedule": Array (3)
  * followers: Array (1)
  * createdAt: ISODate("2026-09-30T18:30:47.056+00:00") September 30, 2026 at 6:30:47 PM UTC
  * updatedAt: ISODate("2026-09-30T18:35:29.654+00:00") September 30, 2026 at 6:35:29 PM UTC
  __v: 1
}

{
  "_id": ObjectId("6abe1b155139980c48f4d57"),
  "title": "Weight Loss Starter",
  "description": "6 week plan mixing cardio and light strength",
  "category": "weight loss",
  "durationWeeks": 6,
  "createdBy": ObjectId("6abd46763cabd3c7c9b9ca30"),
  "schedule": Array (2)
  * followers: Array (2)
  * createdAt: ISODate("2026-10-01T08:34:25.047+00:00") October 1, 2026 at 8:34:25 AM UTC
  * updatedAt: ISODate("2026-10-01T08:34:25.047+00:00") October 1, 2026 at 8:34:25 AM UTC
}

{
  "_id": ObjectId("6abe1b155139980c48f4d58"),
  "title": "Muscle Gain Program",
  "description": "8 week strength plan for building muscle",
  "category": "strength",
  "durationWeeks": 8,
  "createdBy": ObjectId("6abe19cc55139980c48f4d38"),
  "schedule": Array (3)
  * followers: Array (3)
  * createdAt: ISODate("2026-10-01T08:34:25.047+00:00") October 1, 2026 at 8:34:25 AM UTC
  * updatedAt: ISODate("2026-10-03T18:01:47.240+00:00") October 3, 2026 at 6:01:47 PM UTC
  __v: 1
}

{
  "_id": ObjectId("6abe1b155139980c48f4d59"),
  "title": "Endurance Builder",
  "description": "8 week plan to improve stamina",
  "category": "endurance",
  "durationWeeks": 8,
  "createdBy": ObjectId("6abe19cc55139980c48f4d39"),
  "schedule": Array (2)
  * followers: Array (1)
  * createdAt: ISODate("2026-10-01T08:34:25.047+00:00") October 1, 2026 at 8:34:25 AM UTC
  * updatedAt: ISODate("2026-10-01T08:34:25.047+00:00") October 1, 2026 at 8:34:25 AM UTC
}

```

Metrics:

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

localhost:27017

- RailwayDB
- RailwayDB
- admin
- campusDB
- config
- fittrack
 - metrics
 - plans
 - users
 - workouts
- local
- socialMedia

localhost:27017 > fittrack > metrics

Documents 20AggregationsSchemaIndexes1Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

251 - 20 of 20

Explain

Reset

Find

Options

```
_id: ObjectId('6ab1769a13876cca94ec9c')
user: ObjectId('6ab48383cabd37c9b9ca31')
weight: 60
height: 165
bmi: 22
date: ISODate('2026-10-01T08:18:49.796+00:00') October 1, 2026 at 8:18:49 AM UTC
createdAt: ISODate('2026-10-01T08:18:49.796+00:00') October 1, 2026 at 8:18:49 AM UTC
updatedAt: ISODate('2026-10-01T08:18:49.796+00:00') October 1, 2026 at 8:18:49 AM UTC
__v: 0
```

```
_id: ObjectId('6ab1afc55139980c48f4d47')
user: ObjectId('6ab4d6763cabd37c9b9ca30')
weight: 64
height: 165
bmi: 23.5
date: ISODate('2026-08-01T08:00:00.000+00:00') August 1, 2026 at 12:00:00 AM UTC
createdAt: ISODate('2026-10-01T08:34:03.999+00:00') October 1, 2026 at 8:34:03 AM UTC
updatedAt: ISODate('2026-10-01T08:34:03.999+00:00') October 1, 2026 at 8:34:03 AM UTC
```

```
_id: ObjectId('6ab1afc55139980c48f4d48')
user: ObjectId('6ab4d6763cabd37c9b9ca30')
weight: 62.5
height: 165
bmi: 23
date: ISODate('2026-08-15T08:00:00.000+00:00') August 15, 2026 at 12:00:00 AM UTC
createdAt: ISODate('2026-10-01T08:34:03.999+00:00') October 1, 2026 at 8:34:03 AM UTC
updatedAt: ISODate('2026-10-01T08:34:03.999+00:00') October 1, 2026 at 8:34:03 AM UTC
```

```
_id: ObjectId('6ab1afc55139980c48f4d49')
user: ObjectId('6ab4d6763cabd37c9b9ca30')
weight: 61
height: 165
bmi: 22.4
date: ISODate('2026-09-01T08:00:00.000+00:00') September 1, 2026 at 12:00:00 AM UTC
createdAt: ISODate('2026-10-01T08:34:03.999+00:00') October 1, 2026 at 8:34:03 AM UTC
updatedAt: ISODate('2026-10-01T08:34:03.999+00:00') October 1, 2026 at 8:34:03 AM UTC
```

```
_id: ObjectId('6ab1afc55139980c48f4d4a')
user: ObjectId('6ab4d6763cabd37c9b9ca30')
weight: 60
height: 165
bmi: 22
date: ISODate('2026-09-20T08:00:00.000+00:00') September 20, 2026 at 12:00:00 AM UTC
createdAt: ISODate('2026-10-01T08:34:03.999+00:00') October 1, 2026 at 8:34:03 AM UTC
updatedAt: ISODate('2026-10-01T08:34:03.999+00:00') October 1, 2026 at 8:34:03 AM UTC
```

Database Overview:

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

localhost:27017

- RailwayDB
- RailwayDB
- admin
- campusDB
- config
- fittrack
 - metrics
 - plans
 - users
 - workouts
- local
- socialMedia

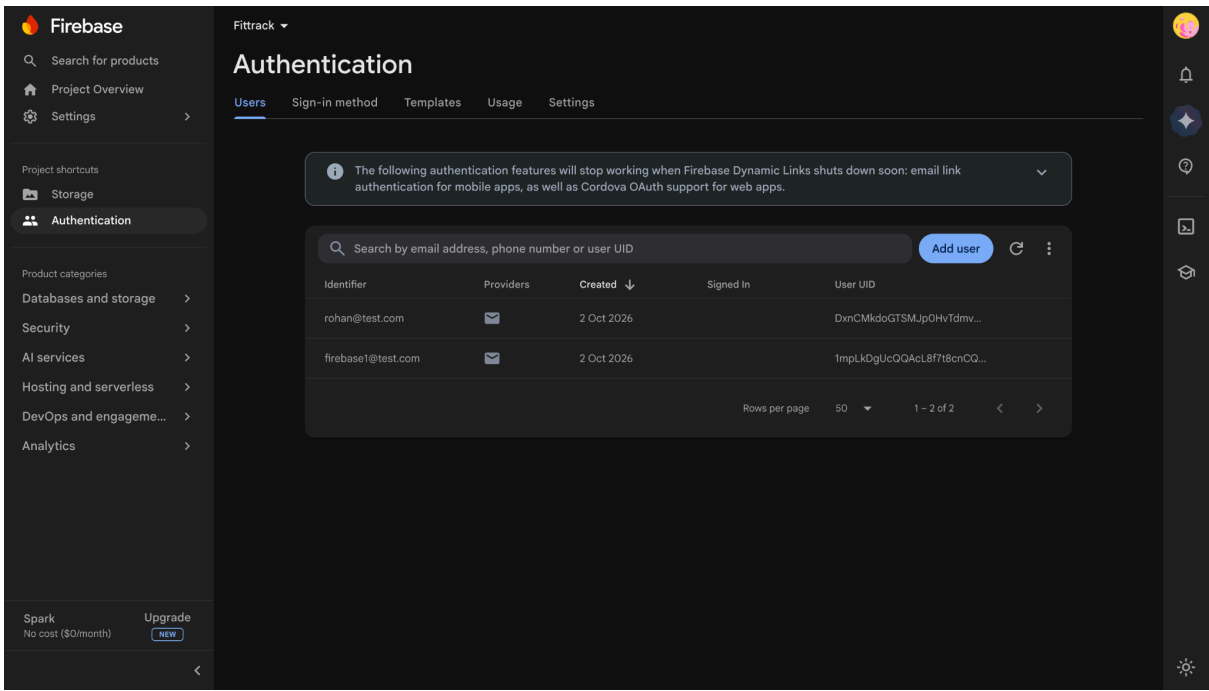
localhost:27017 > fittrack

Create collection

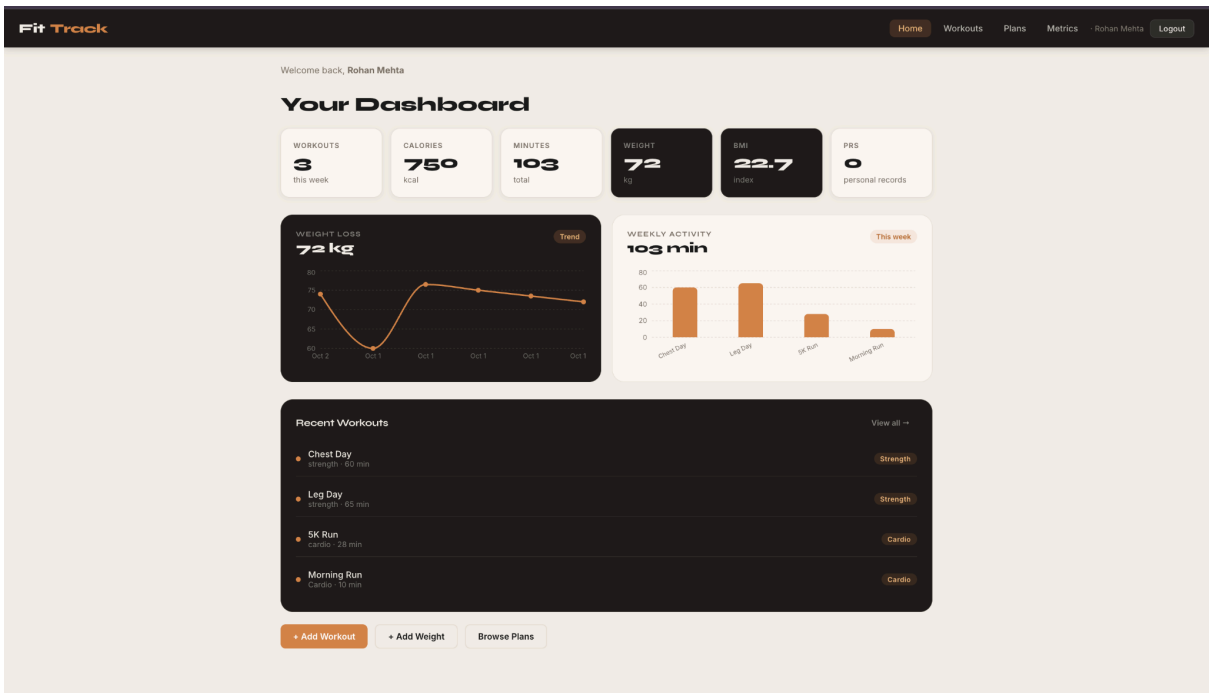
Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
metrics	-	36.86 kB	2.69 kB	20	134.00 B	1	36.86 kB
workouts	-	36.86 kB	3.31 kB	14	236.00 B	1	36.86 kB
users	-	36.86 kB	1.64 kB	7	234.00 B	2	73.73 kB
plans	-	36.86 kB	2.21 kB	5	442.00 B	1	36.86 kB

Firebase Snapshots



Frontend UI Snapshots



Fit Track

HomeWorkoutsPlansMetricsRohan MehtaLogout

Workouts

Track your training sessions

Add a Workout

TITLE

e.g. Morning Run

TYPE

Cardio

DURATION (MIN)

30

+ Add

All Workouts

4 sessions

C

Morning Run

Cardio10 min

Delete

DT

5K Run

Cardio28 min

Delete

DT

Leg Day

Strength65 min

Delete

DT

Chest Day

Strength60 min

Delete

Fit Track

HomeWorkoutsPlansMetricsRohan MehtaLogout

Plans

Create and follow workout plans

Create a Plan

PLAN TITLE

e.g. 30-Day Fat Burn

CATEGORY

Beginner

+ Create Plan

Search Plans

KEYWORD

e.g. cardio, strength...

SearchReset

All Plans

5 plans

PL

Weight Loss Starter

Weight Loss2 followers

+ Follow

PL

Muscle Gain Program

Strength3 followers

+ Follow

PL

Endurance Builder

Endurance1 follower

+ Follow

PL

Flexibility and Core

Flexibility0 followers

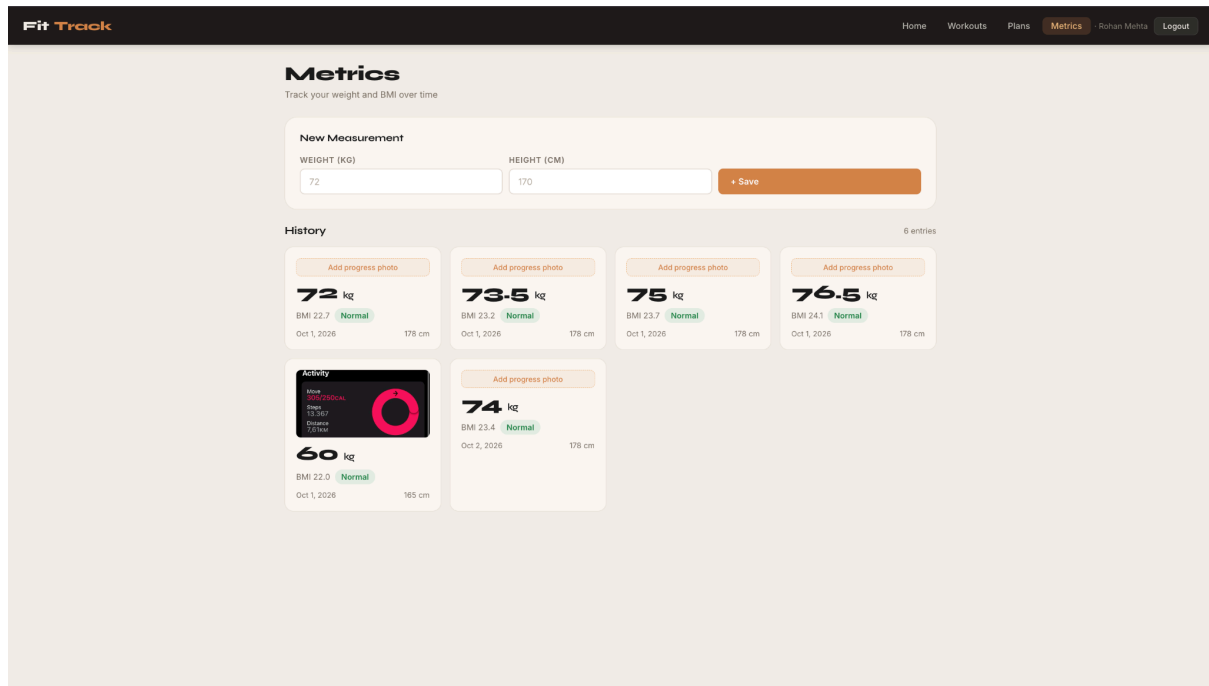
+ Follow

PL

Beginner Strength Plan

Strength1 follower

+ Follow



7. Conclusion

FitTrack demonstrates a complete, production-patterned backend built from the ground up using modern Node.js tools. The project satisfies every academic deliverable and goes beyond the base requirements in several areas:

Authentication is handled via a dual system—JWT for all API routes, with Firebase token also accepted on the photo upload endpoint. Both Firebase Auth and MongoDB stay in sync on every register and login through the `syncFirebaseUser()` function, which looks up or creates the Firebase user and stores the UID back in MongoDB.

Full CRUD operations are implemented across four data domains—Users, Workouts, Plans, and Metrics—with strict ownership enforcement at the controller level. Any attempt to read or modify another user's data returns HTTP 403 Forbidden.

BMI calculation is performed server-side from the submitted weight and height values, stored to one decimal place, and returned to the client—it is never accepted as input from the user.

Analytics aggregation produces weekly and monthly summaries on demand using Mongoose queries and a single-pass in-process loop—no aggregation pipeline or external library required.

File upload flows from browser `FormData` through multer's memory storage to a local write and a stored URL—progress photos are served back as static files via Express.

Push notifications use Firebase Cloud Messaging. A device registers its FCM token once, and the reminder endpoint fires a live push notification on demand.

API documentation is provided via the Postman collection and environment in docs/, covering all endpoints.

The React frontend integrates all backend features with Recharts data visualisations, per-page form validation, toast notifications, and an auth-aware navbar that stays in sync across browser tabs without a page reload.

The modular MVC architecture—routes to middleware to controllers to models—keeps every concern isolated and makes the system straightforward to extend with new features or replace individual components independently.

8. References

- Node.js Documentation — <https://nodejs.org/en/docs/>
- Express.js Official Documentation — <https://expressjs.com/>
- Mongoose ODM Documentation — <https://mongoosejs.com/docs/>
- Firebase Admin SDK — Node.js Setup — <https://firebase.google.com/docs/admin/setup>
- Firebase Cloud Messaging — <https://firebase.google.com/docs/cloud-messaging>
- JSON Web Tokens — jwt.io — <https://jwt.io/introduction>
- bcryptjs on npm — <https://www.npmjs.com/package/bcryptjs>
- express-validator Documentation—<https://express-validator.github.io/docs/>
- multer—Multipart File Upload Middleware—<https://github.com/expressjs/multer>
- React Documentation — <https://react.dev/>
- React Router v7 — <https://reactrouter.com/>
- Recharts — Chart Library for React — <https://recharts.org/>
- MongoDB Documentation — <https://www.mongodb.com/docs/>

9. GitHub Link: <https://github.com/ishitanakhawa/Backend---MajorProject.git>